

ScaleLink

A Beginner Developer's Architectural Guide, Component Breakdown, and Presentation Pitch

System Name:	ScaleLink High-Throughput URL Engine
Architecture:	Go 1.22, Nginx, Redis 7, PostgreSQL 16, Prometheus, Grafana
Verified Scale:	12,410 Requests/sec Median Latency: 1.2 ms
Cache Efficiency:	96.8% Hit Ratio (Redis Cache-Aside + Negative Caching)
Target Audience:	Clients, Teachers, Technical Interviewers (FAANG Depth)

What this document gives you:

Even if you are a junior or beginner developer who only knows the basics of URL shortening (pasting a link to make it shorter), this document explains **every single advanced component** in simple, real-world analogies. By reading this guide, you will be able to explain the entire system confidently, defend every design choice, and impress any client, professor, or senior interviewer.

Part 1: Why ScaleLink Exists (The Problem)

A typical beginner project creates a URL shortener with just one server and a database. When a user pastes a URL, it stores the code in the database. When someone visits the short code, it looks up the code in the database and redirects the user.

That works fine for 5 friends. But it collapses in production.

■ **The Black Friday Disaster:**

Suppose a retailer tweets a short link to 10 million followers. Within 1 minute, 500,000 people click the link. If every click queries the database:

- 1. Database connection limits (default 100) are exceeded in 1.5 seconds.
- 2. Database CPU spikes to 100%, causing queries to queue up and time out.
- 3. The server crashes with HTTP 500 / 502 errors. The retailer loses sales.
- 4. Attackers spam random invalid codes, forcing full disk table scans that keep the database dead.

ScaleLink is built to solve this problem permanently. It uses high-performance Redis caching, non-blocking streams, atomic token-bucket rate limiting, and an Nginx load balancer to sustain over **12,400 clicks every single second** with a **1.2 millisecond response time**.

Part 2: The 5 Endpoints & Working URLs Explained

URL & Port	What It Is (Simple Analogy)	Why It Exists & How It Works
1. Web Dashboard http://localhost:8080	The Front Door / Storefront	The user-friendly React web interface where users paste long links, specify optional custom aliases, click 'Shorten', copy short URLs, download QR codes, and view click analytics graphs.
2. Backend Health http://localhost:8080/health	The Doctor's Stethoscope / Pulse	A lightning-fast diagnostic endpoint. Cloud orchestrators (AWS / Kubernetes) ping this every 5 seconds. If PostgreSQL or Redis crashes, this reports it so the cloud can restart the container before users notice.
3. Prometheus Metrics http://localhost:8080/metrics	The Airplane Black Box / Digital Ledger	A live raw data feed. Every single HTTP request, Redis cache hit, stream event, or 429 rate limit block increments an internal counter here in standardized plain text.
4. Prometheus UI http://localhost:9090	The Inspector / Data Collector	Prometheus scrapes all backend nodes every 5 seconds, saves the history into a specialized time-series database, and lets engineers execute queries (e.g. 'What was our peak QPS over the last hour?').

5. Grafana Dashboard http://localhost:3000	NASA Mission Control / Car Speedometer	Raw numbers are unreadable to humans. Grafana turns Prometheus data into live dials, gauges, and colored graphs showing throughput (12,410 req/s), 96.8% cache hit ratio, and latency percentiles.
--	---	--

Part 3: The 5 Engineering Superpowers (How It Works)

1. Redis Cache-Aside (The Desk vs. Library Shelf)

The Analogy: Querying a PostgreSQL database on a hard drive is like walking to a library, finding the right aisle, and pulling a book off the shelf (takes 10 to 20 milliseconds). Reading from Redis in RAM memory is like having the book open directly on your desk (takes 1 millisecond).

In ScaleLink: When someone clicks a link for the first time, ScaleLink fetches it from PostgreSQL and places a copy in Redis. The next 100,000 visitors get redirected directly from Redis RAM in **1.2 milliseconds**. The database is completely protected.

2. Negative Caching (The Anti-Hacker Sticky Note)

The Analogy: Imagine a prankster calls a store 500 times asking for a product that doesn't exist. If the clerk walks to the storage room every time, real customers get ignored. A smart clerk puts a sticky note on the phone: *'We don't have item #999, stop checking the back room.'*

In ScaleLink: When attackers spam random invalid codes to force database scans (Cache Penetration), ScaleLink writes a key `link:neg:{code}` into Redis with a 5-minute expiry. All subsequent fake requests return 404 from memory in 1ms without touching PostgreSQL.

3. Token-Bucket Rate Limiter (Preventing Bot Spam)

The Analogy: Picture a bucket holding 20 tokens. Every time you shorten a link, you spend 1 token. A water faucet drips 1 new token into your bucket every few seconds. If a bot tries to create 100 links in 1 second, the bucket empties instantly and the bot receives HTTP 429 Too Many Requests with a Retry-After countdown.

In ScaleLink: Built with an atomic Redis Lua script. It runs inside Redis memory in 0.3ms, ensuring accurate limits across all servers simultaneously.

4. Redis Streams & Click Worker (Non-Blocking Analytics)

When a user clicks a link, they want to reach their destination **instantly**. They do not want to wait while the server geolocates their IP, parses their browser User-Agent, and writes a log into PostgreSQL.

In ScaleLink: The redirect handler drops a lightweight event onto an in-memory conveyor belt (**Redis Streams**) in 0.2ms and immediately redirects the user. A separate background **Click Worker** pulls 100 events at a time and writes them to PostgreSQL in one fast batch. Even if 5,000 people click simultaneously, the user experience is instant.

5. Nginx Load Balancing Across 3 API Instances

ScaleLink runs 3 identical Go API instances (`api1`, `api2`, `api3`). Nginx sits at the edge and uses the `least_conn` algorithm: incoming traffic is automatically routed to whichever server is currently handling the fewest active connections.

Part 4: Step-by-Step Life of a Link (Execution Flow)

Phase	What Happens Behind the Scenes
Step 1: Link Creation	1. User submits long URL via POST /api/links. 2. ScaleLink validates URL format and rejects malicious loopback IPs (SSRF protection). 3. Cryptographic Base62 algorithm generates a 6-character code (e.g. aB3x9Q). 4. Stored in PostgreSQL; returns HTTP 201 Created.
Step 2: The First Click (Cold Cache Miss)	1. Browser requests GET /aB3x9Q. 2. Nginx routes request to api1 via least_conn. 3. api1 checks Redis RAM (Cache Miss). Reads PostgreSQL. 4. Stores link in Redis with 1-hour TTL. 5. Emits click event to Redis Streams (non-blocking). 6. Returns HTTP 302 Found redirect to browser.
Step 3: Subsequent Clicks (Hot Cache Hits)	1. Thousands of users click the link. 2. Nginx distributes traffic across api1, api2, and api3. 3. Redis already has the link in RAM (Cache Hit). 4. PostgreSQL is never touched! Redirect executes in 1.1 ms. 5. Response includes headers: X-Cache: HIT, X-Served-By: redis.
Step 4: Background Analytics	1. Click Worker consumes 100 events from Redis Stream. 2. Parses device type (mobile/desktop/tablet) and country. 3. Inserts batch into monthly-partitioned database table in a single 18ms transaction. 4. Worker issues XACK only after DB commit succeeds (Zero Event Loss).
Step 5: Telemetry Display	1. Prometheus scrapes /metrics every 5 seconds. 2. Grafana calculates live throughput, cache ratios, and latencies. 3. Dashboard updates live on http://localhost:3000.

Part 5: How to Pitch ScaleLink to Teachers & Clients

The 30-Second Elevator Pitch

*"I built **ScaleLink**, an enterprise-grade distributed URL shortening and real-time click analytics platform engineered for extreme read traffic. While a standard URL shortener crashes when hundreds of thousands of users click simultaneously, ScaleLink uses a multi-tier architecture with Redis cache-aside, negative caching, atomic Lua rate limiting, an Nginx least-connections cluster, and an asynchronous Redis Streams analytics pipeline.*

*During load testing with 300 concurrent users, the cluster handled over **12,400 requests per second** with a median response time of **1.2 milliseconds** and a 96.8% cache hit ratio, fully monitored via live Prometheus and Grafana dashboards."*

Part 6: Top Questions You Will Be Asked & Winning Answers

Q1: Why did you use Redis Streams instead of writing clicks directly to the database?

Your Answer: Writing to PostgreSQL takes 10 to 30ms because it writes to disk, updates indexes, and manages locks. Making the user wait for that on every click slows down the redirect. By pushing the click event to Redis Streams in RAM (0.2ms), the user is redirected instantly. Our background worker then batches 100 writes into PostgreSQL at once, cutting database load by 90%.

Q2: What is 'Negative Caching' and why does ScaleLink need it?

Your Answer: If attackers or bots spam millions of non-existent shortcodes, a normal cache misses every time and sends the query to PostgreSQL. This 'Cache Penetration' attack quickly exhausts database connections. ScaleLink caches non-existent codes in Redis for 5 minutes (link:neg:{code}). Subsequent bogus requests return 404 from RAM in 1ms without touching PostgreSQL.

Q3: Why did you choose the least_conn load balancing algorithm in Nginx?

Your Answer: Round-robin sends traffic in a simple circle regardless of server load. If one server gets slowed down by a heavy request, round-robin keeps piling work onto it. least_conn dynamically routes each new request to whichever of the 3 API instances currently has the fewest active connections, preventing latency spikes.

Q4: What happens if the background worker crashes in the middle of processing?

Your Answer: Zero event loss. We use Redis Consumer Groups. When the worker pulls events, Redis marks them as pending. The worker only acknowledges (XACK) to Redis after PostgreSQL commits the database transaction. If the worker crashes, unacknowledged events remain in the Pending Entries List (PEL) and are safely reprocessed upon restart via XAutoClaim.

Q5: How did you measure and verify the 12,000+ QPS benchmark?

Your Answer: We used k6 distributed load testing scripts in /loadtest ramping up to 300 virtual users across warm cache reads, cold cache misses, and burst rate-limiting scenarios. All metrics were captured by Prometheus and visualized in Grafana, recording p50, p95, and p99 latency distributions.

Key Numbers Cheat Sheet (Memorize These for Interviews)

Metric	Value	Interview Talking Point
Throughput	12,410 req/s	Sustained across 3 Go instances behind Nginx
Median Latency (p50)	1.2 ms	Sub-2 millisecond redirect on hot links
Tail Latency (p99)	8.6 ms	99% of requests complete under 10ms
Cache Hit Ratio	96.8%	Redis absorbs 97 out of 100 queries
Negative Cache Efficacy	98.7% DB load reduction	Absorbs 404 attack traffic in RAM
Worker Batch Persist	14 - 22 ms	100 analytics rows committed per transaction